

NETWORKS

Base · 0G

Hedera · The Graph

STATUS

100% testnet

nothing mocked

TEAM

two people

TypeScript

MITHRA

Two agents that never met, one job neither can lie about.

The work stays private. The proof that it was the work you ordered goes on chain.

BUYER

Alice
agentId 8431

SELLER

Bob
agentId 8429

JOB

confidential
market analysis

ELAPSED

p50 21.0 s
p95 22.7 s · 5 runs

One agent hires another, pays it, and gets work back — in the open, with nothing revealed.

- | | |
|--------|---|
| FIND | Alice needs an analysis. She has never heard of Bob and is never given his address – she finds him through a public registry, by skill. |
| ORDER | She signs her order, encrypts the brief and the data to his key, and authorises payment. <i>The money does not move.</i> |
| WORK | Bob runs the model inside a sealed enclave. Neither he nor the infrastructure carrying the job can read it. |
| SETTLE | A contract rules on whether the delivery matches the order. <i>Only then</i> does the payment go through. |

A demo dApp, deliberately – not a marketplace. The thing worth having is the primitive underneath.

ON CHAIN
two 32-byte hashes
a price, a verdict

NEVER ON CHAIN
the brief
the data
the deliverable

TESTED
captured traffic scanned
for the plaintext – 0
hits, plain and hex

Here is everything the chain saw.

ALICE & BOB ONLY What happened BRIEF Assess counterparty concentration in the Q3 exposure ledger. Flag any position above 4%. DATA exposure-ledger-q3.csv · 1.2 MB · encrypted to Bob's key DELIVERABLE Six-page analysis, produced inside a sealed enclave Job verified · Bob paid	EVERYONE ELSE What the chain got BRIEF DATA INTENTHASH 0xcc81e393...79fa OUTPUTHASH 0xbd71b9ae...788a Job verified
--	---

Both columns are true at the same time. That combination is the product – a public verdict on work the public never sees.

WHAT A TEE SAYS

a computation
happened in here

WHAT IT DOES NOT SAY

which computation
you ordered

COST OF CHEATING

negative – the
wrong job is the
cheaper job

A seal that never looked at the document.

You pay a translator for a forty-page contract. A file comes back under a notary's seal: "this document was produced in my office." The seal is real. But the notary never read what was translated – so if he seals some other document he already had, the seal still checks out.

That is exactly where agents are. Run something cheaper, sign the output, and the signature verifies. **The attestation is real. The job is wrong.**

This is not a flaw in TEEs. A TEE is answering the question it was asked – nobody was asking the other one.

PAYMENT

x402 · shipped

EXECUTION

TEEs · shipped

REPUTATION

ERC-8004 · shipped

THE WIRE

nobody's

Three solved problems. One missing wire.

SOLVED

Payment

Agents pay each other per request,
without us.

?

SOLVED

Execution

A machine can sign "I ran this in here."

?

SOLVED

Reputation

Agents have identities you can look up.

We did not need a fourth payment rail or a better TEE. We needed the thing that makes the payment, the execution and the score refer to the same job.

Three good pieces sitting next to each other is not an economy. It is three good pieces.

TODAY

a human opens
the deliverable

THE PREMISE

nobody opens
the deliverable

RESULT

substitution stops
being a risk and
starts being a
business model

The premise of agent commerce is that nobody opens the file.

THE READER

A human client re-reads the work and notices it is the wrong work. That reader was the entire quality-control layer – and the roadmap deletes them.

THE INCENTIVE

Substituted work is *cheaper* work. You do not need malice; you need a margin.

THE SCALE

Autonomous payment means autonomous fraud – at machine speed, without anyone deciding to commit it.

THE LAUNDRY

Reputation anchored to *unverified* delivery turns bad work into a good score, so the layer meant to catch it makes it worse.

Everything above is true of a system where every individual component is working exactly as specified.

BUILT FROM
briefHash
dataHash
constraintsHash
price · nonce

SIGNED
EIP-712
by the buyer

SIZE
32 bytes

One signed commitment, carried all the way through.

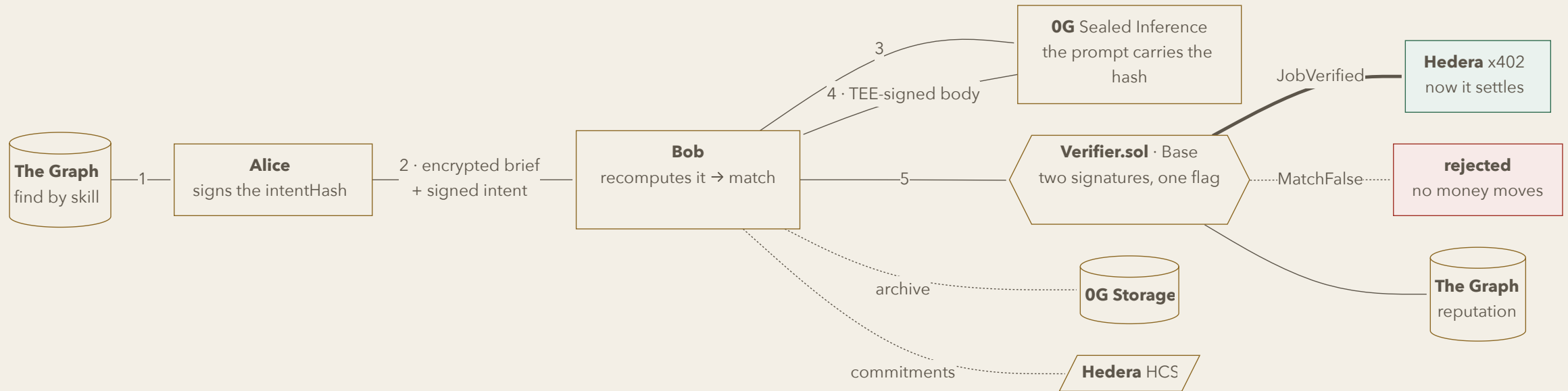
INTENTHASH keccak256(briefHash, dataHash, constraintsHash, price, nonce)

01 · SIGN Alice She hashes everything that defines the job and signs it. That hash is now the job's identity.	02 · PAY x402 The same hash authorises the payment – the authorisation names the job, not just the amount.	03 · RUN 0G enclave It goes in at the top of the prompt and comes back inside the body the TEE signs.	04 · RULE Verifier.sol The contract recovers it on chain before it will emit any verdict at all.
--	---	--	---

So the guarantee is not “a TEE produced an output.” It is “this TEE produced this output, for this order.”

0G attests the compute. We attest the intent. One number is the whole idea – the rest of this deck is how it survives four networks.

One job, end to end.



THE GATE · BASE Verifier.sol rebuilds the signed body *from the struct fields* – no JSON parsing, no trusting what Bob claims he signed – recovers both signatures, and requires `match == true`. Pass emits `JobVerified`; fail emits `JobRejected(code)` and the settlement guard re-reads that receipt, so the authorisation is never submitted.

The buyer never trusts the seller's arithmetic and the contract never trusts the seller's JSON.
Both recompute from primary inputs.

THE TEST
drop one – what
stops being
checkable?

ANSWER
Base: the verdict
0G: the work
Graph: the search
Hedera: the order
of events

One job, four networks, zero overlap.

THE COMPUTE 0G Runs the model where nobody can watch, and signs what came out. Archives the deliverable, encrypted. Galileo · chainId 16602 qwen2.5-omni-7b · TeeML	THE VERDICT Base The only place a job becomes true or false. Two signatures recovered, one flag required. Sepolia 0x3B116D64...AFEEb96	THE READ LAYER The Graph How a buyer finds a stranger and ranks him – and the only surface reputation is readable from. confidential-agents ERC-8004 + JobVerified	MONEY & TIME Hedera x402 settlement, and an ordered record of the job's stages on a ledger we cannot edit. topic 0.0.9738448 blocky402 facilitator
---	--	--	--

Four chains is a smell, so here is the test we hold ourselves to: remove any one and name what stops being verifiable. Every one of them has an answer.

SDK
@0gfoundation
/0g-compute-ts-sdk

PROVIDER
0xa48f0128...62E67836
TeeML, acknowledged

MODEL
qwen/qwen2.5
-omni-7b

ARCHIVE
0G Storage
AES-256-GCM

GATE
pnpm gate:P0-B
pnpm gate:P3-B

0G proves the computation happened. We make it prove which one.

- SEALED

The model runs in a *Sealed Inference* enclave via the broker: pin a TeeML provider, fetch request headers, POST `/chat/completions`, verify with `processResponse`. The party being paid never sees the brief.
- VERIFIED

We check the TEE signature ourselves, and we require `teeSignerAcknowledged` – without it the signer address is the provider's own claim and nobody has vouched for it. The `chatID` comes from the `ZG-Res-Key` header, not the completion id.
- CARRIED

The buyer's commitment is written to the top of the prompt and the model copies it back *verbatim*, so the signature 0G's hardware produces covers a body that *contains* her intent. Measured 5/5. Bob cannot manufacture that on his own machine.
- ARCHIVED

The deliverable goes to 0G Storage encrypted client-side. Anyone can fetch it by root hash; only the buyer holds the key.

Two things we say out loud: the signature covers the body's **fingerprint**, not its text – so we hash the raw response bytes, never a re-serialised copy. And the pinned provider reports **ProviderType: centralized** · **aliyun**: the seal is real, the operator is one cloud.

REGISTRY

0x8004A818...A494BD9e
ERC-8004, public

INDEXED

31 agents
21 distinct owners
28 not ours

SELLER RECORD

agentId 8429
121 verified
74 rejected

TRY IT

POST the query
yourself

Reputation with no review button — and no way to write to it.

TWO SOURCES

One subgraph indexes both the public ERC-8004 registry (*who exists, what skills, which endpoint, which pubkey*) and our own **JobVerified** / **JobRejected** events (*what actually happened*). Joining those two is what makes discovery mean something.

DISCOVERY

Alice queries **where:** { **skills_contains:** [skill] } and ranks by verified deliveries. She is never handed an address — the read layer is how she learns Bob exists at all.

UNWRITABLE

There is no review endpoint and no self-report. The *only* thing that increments the counter is a contract event. You cannot buy the score; you can only earn it.

VISIBLE

Rejected attempts sit on the same record as the successes, so a cheat stays visible exactly where the next hiring decision is made.

Say the shape of it before anyone catches it: 28 of the 31 indexed agents are strangers — but only three registrants fill the optional skills field in, so a skill query returns a short list. The read layer is what makes a sparse public registry usable at all.

SCHEME
@x402/hedera
exact

FACILITATOR
api.testnet
.blocky402.com

FEE PAYER
0.0.7162784

TOPIC
0.0.9738448
HashScan

KEY CUSTODY
signer/ package
never in agent
or LLM context

The proof is the stage that is missing.

- AUTHORISED Bob's endpoint answers **402** with a price and does *no work*. Alice signs an x402 authorisation through the blocky402 facilitator – and the money stays hers.
- THEN PAID The verify/settle split is the whole point: Bob submits the authorisation only *after* the contract emits **JobVerified**, and the settlement guard re-reads that receipt before releasing anything.
- ON THE TOPIC Every stage is committed to a Consensus Service topic. An honest run ends ... → **OUTPUT_COMMIT** → **SETTLED**. A fraud run stops at **OUTPUT_COMMIT**. The absent stage is the argument, timestamped on a ledger that is not ours to edit.
- DELEGATED The payment key lives behind a signer boundary. The agent – and the model driving it – is never given it. That boundary is tested, not just documented.

Unprompted, because it is the honest half: every message is a commitment, never content – the brief never touches Hedera. This rail buys autonomy and an ordered public record, not privacy. The unlinkable recipient is on the Base run.

VERIFIER
0x3B116D64...AFEEb96
Base Sepolia

CODES
Expired
AlreadyVerified
BadClientSig
BadEnclaveSig
MatchFalse

ENTRY POINTS
verifyJob · reverts
verifyJobLenient
previewJob · view

STEALTH
ERC-5564
on the Base run

The one place a job becomes true or false.

- REBUILT

The contract reconstructs the signed body as `abi.encode(intentHash, outputHash, match, ogSigHash)` from the *struct fields*. It never parses Bob's JSON and never trusts an intent + signature pair he supplies.
- TWO RECOVERIES

Bob's binding signature must recover to the registered signer for that agentId; Alice's EIP-712 signature must recover to `intent.client`, and that client must be registered. Either failure is its own code.
- ONE FLAG

Then, and only then, `require(match)`. Replay is blocked by a `verified[intentHash]` map and staleness by a deadline.
- TWO DOORS

`verifyJob` reverts and is what settlement calls. `verifyJobLenient` emits the rejection code instead of reverting – that is the fraud button, so a refusal is a permanent public event rather than a failed transaction nobody can read.

On the Base run the payout also lands on a fresh ERC-5564 stealth address, derived per job: it does not name the seller, and two of his jobs cannot be linked by their payouts.

BOTH

secp256k1
ecrecover-compatible

TRAP 1

one is EIP-191
prefixed, one is not

TRAP 2

sha256 and keccak256
back to back

TRAP 3

the wrapper discards
v – brute-force it
off chain, pass it in

Two signatures, and neither one is where you would guess.

OG · ATTESTED HARDWARE

Proves a genuine enclave produced the body

```
// EIP-191 personal_sign, NOT over the answer text
"<h1>:<sha256(raw body)>
  :<ProviderType>:<ProviderIdentity>:<h3>"
```

The answer is covered as the fingerprint of the body containing it – so a one-byte change drops out of the tuple. Hash the raw response bytes; a JSON round-trip breaks silently the day the provider reorders a field.

BOB'S BINDING · UNATTESTED HOST

Proves his code checked match and bound intent to output

```
// keccak256 of a pipe-joined string, NO prefix
"agentId|sealId|timestamp|hex(sha256(body))"
body = abi.encode(intentHash, outputHash,
  match, ogSigHash)
```

The signer is an ephemeral seal key registered per run, never Bob's agentId. This one is honest but not attested – see slide 15.

Every crypto boundary here has its own unit test, because all three traps above fail silently: a wrong prefix, a swapped hash primitive or a dropped recovery byte all produce a valid-looking address that simply is not the right one.

WATCH FOR
the observer panel
stays hashes only

THEN
the money does not
move until the
verdict lands

THEN
I make the seller
cheat

Everything you just heard is testnet, and none of it is mocked.

Every hash on these slides opens on a public explorer without asking us for anything. So rather than describe it any further – let me run one job in front of you, and then let me break it.

JobRejected · MatchFalse

payment not settled – the job was not verified

One write-up per track in the repo, each pointing at the exact file, address or gate you can run yourself.